
Template de dashboard analítico desarrollado íntegramente con IA

Trabajo Fin de Máster del Máster de desarrollo con IA. Un producto real, desplegado en producción, y el método usado para construirlo.

DEMO EN VIVO

tfm-master-desarrollo-ia.sergiogdev.com



QUIÉN SOY

Sergio García

Desarrollador de Software FullStack

WEB

sergiogdev.com

Trabajo como desarrollador Senior en Innusual y, de vez en cuando, saco tiempo para proyectos propios como freelance.

Me muevo por curiosidad: cuando aparece algo que no sé hacer, quiero aprender a hacerlo bien. Este máster viene justo de ahí, de querer entender a fondo cómo se construye software con IA en lugar de quedarme en usarla a medias.

NexDash sale del mismo sitio. Casi todos los proyectos que empiezo piden la misma base antes de poder arrancar de verdad, así que esta vez decidí construirla una sola vez y hacerlo bien.

Casi todos mis proyectos necesitan lo mismo, y esa base se rehace desde cero cada vez

Navegación, autenticación, tablas, gráficas, formularios, estados de carga y error: el mismo conjunto de piezas, reconstruido en cada nuevo proyecto porque no existía un punto de partida real del que arrancar.

LO QUE SE REPITE EN CADA PROYECTO

- Navegación y layout con sidebar, breadcrumbs y topbar
- Autenticación con sesión, cookies y rutas protegidas
- Tablas con búsqueda, filtros y paginación
- Gráficas, KPIs y formularios validados
- Estados de carga, error y vacío en cada sección

EL COSTE DE NO TENERLO

Cada proyecto nuevo empieza pagando el mismo peaje: reconstruir la base antes de poder tocar el problema real. Una maqueta que se ve bien en una captura no lo resuelve. Lo que hace falta es algo que se clone y se extienda sin desmontarlo antes.

Dos objetivos independientes, y encajan

El máster pide construir algo genuinamente útil desarrollándolo íntegramente con IA. Yo necesito un template real del que partir. No son el mismo objetivo, pero comparten el mismo criterio de éxito.

Objetivo del máster

No una demostración de que un modelo puede escribir código, sino un proyecto real llevado hasta el final (arquitectura, tests, CI y despliegue en producción) con desarrollo asistido por IA como forma de trabajo.

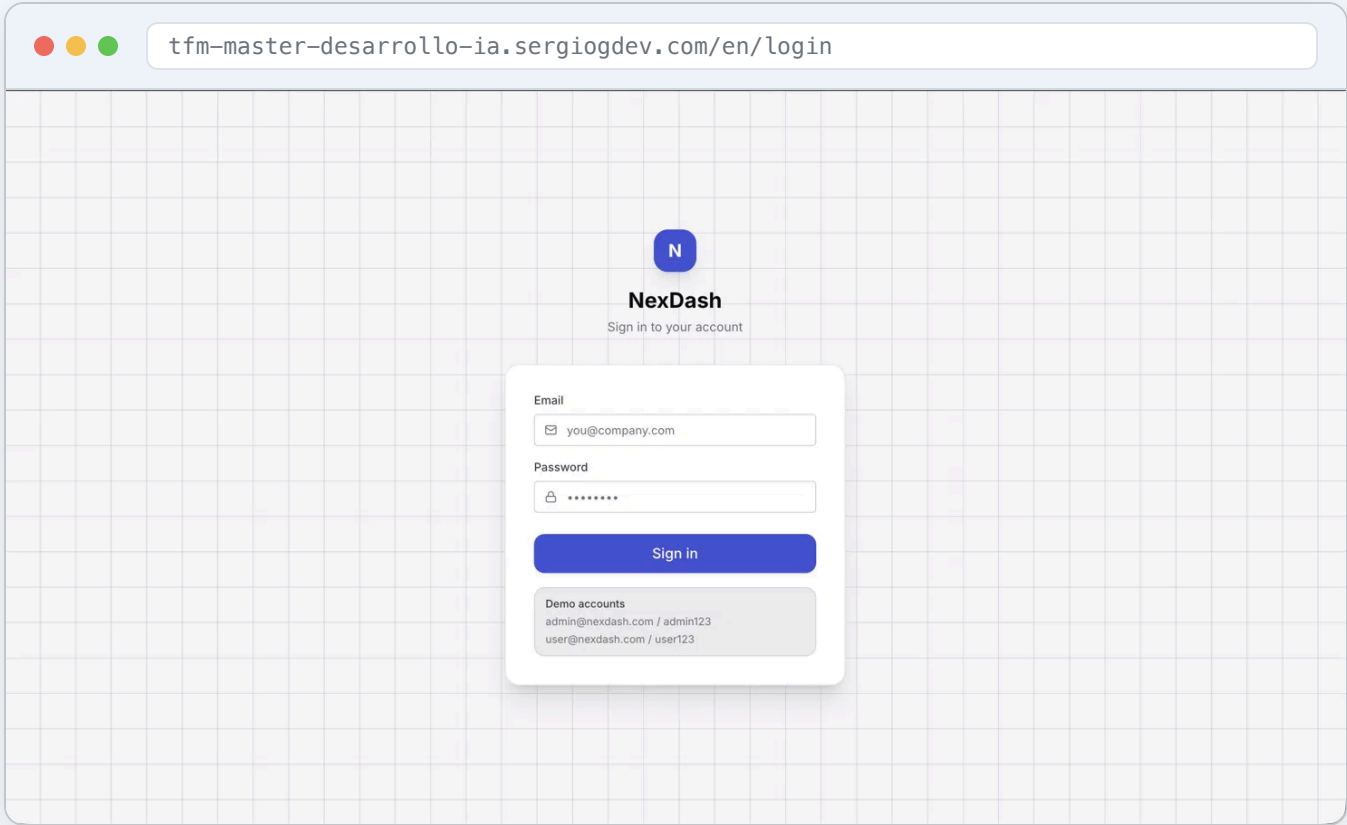
Objetivo personal

Construir un template de dashboard funcional y real del que partir en los siguientes proyectos. Algo que se clona y se extiende, no una maqueta que se ve bien en una captura.

Por qué encajan: la calidad de un template no se mide por lo que hace, sino por lo bien que se puede extender. Eso obliga a resolver exactamente lo que un proyecto de demostración puede permitirse ignorar: convenciones verificables, fronteras explícitas entre capas, y un registro de qué se descartó y por qué.

La aplicación: autenticación con cookies HttpOnly

Login funcional contra un backend mock, con la misma disciplina de validación y manejo de errores que tendría uno real. La sesión nunca es accesible desde el JavaScript del cliente.



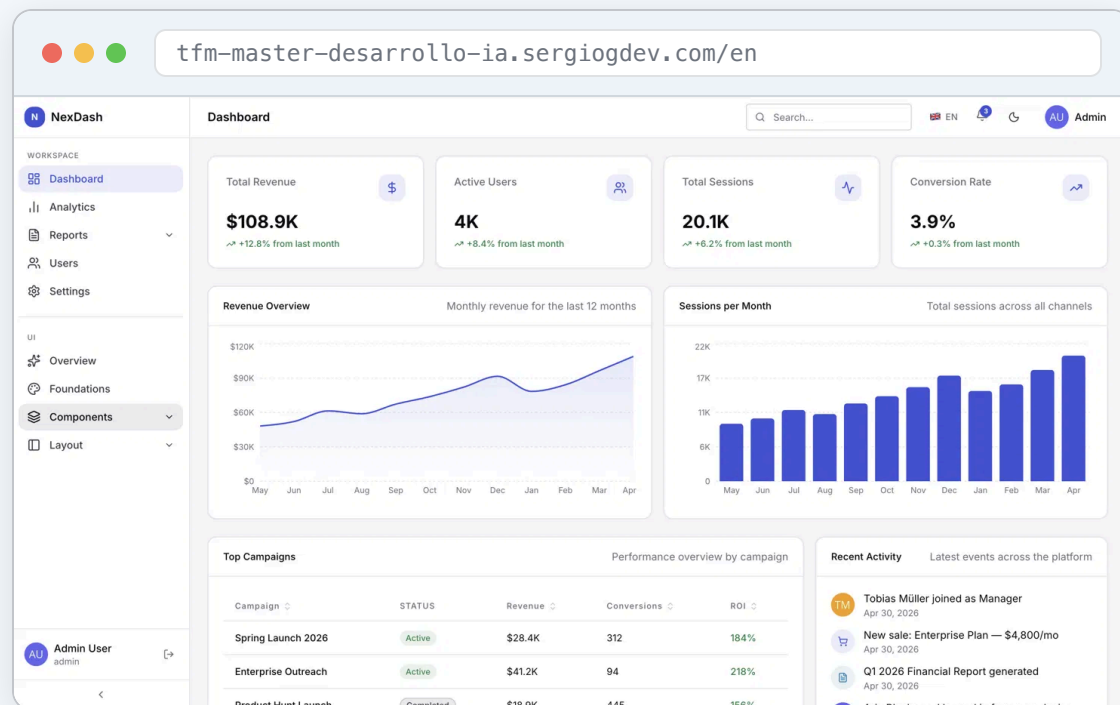
CREDENCIALES DE LA DEMO

EMAIL	PASSWORD	ROL
admin@nexdash.com	admin123	admin
user@nexdash.com	user123	user

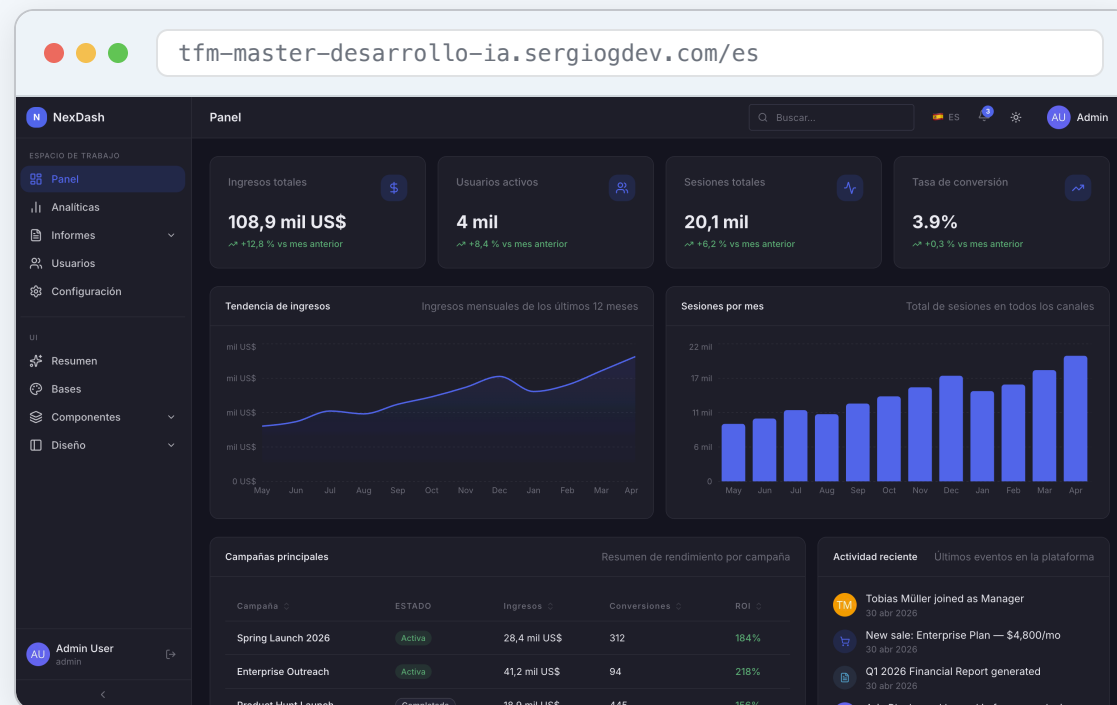
- **Cookie HttpOnly:** nexdash_session, nunca accesible al JS del cliente ni guardada en localStorage
- **Validación en frontera:** toda respuesta de /auth/* pasa por Zod antes de salir del handler
- **Token opaco:** un UUID de servidor; el cliente no lo lee ni lo inspecciona

No es solo traducir texto: el locale cambia el formato, el tema cambia por tokens

Mismo componente, mismos datos, cero ramas de código. A la izquierda, inglés en tema claro; a la derecha, español en tema oscuro, misma build y misma marca de acento.



Tema claro · Inglés



Tema oscuro · Español

- **Formato, no solo texto:** \$108.9K se convierte en 108,9 mil US\$, con coma decimal: routing por prefijo de URL (/en, /es) y detección automática vía Accept-Language
- **Dos temas y seis acentos,** todos como CSS custom properties, ningún color hardcodeado en componentes

El proyecto en cifras

Recontadas contra el repositorio en el momento de generar este documento, no son estimaciones.



Desplegado en producción con TLS (Let's Encrypt) y auto-deploy: cada push a main que pasa las cinco puertas de CI dispara el build y el despliegue en el VPS sin intervención manual.

Cada feature es dueña de su capa de datos completa

No hay una carpeta mocks/ centralizada. Fixtures, handler, query keys, hooks, schemas, tipos y traducciones viven dentro del propio feature.

```
src/features/{name}/
├─ api/
│   ├── _mock-data.ts      # fixtures privadas
│   ├── {name}.handler.ts  # branch en USE MOCKS
│   ├── {name}.keys.ts     # query keys, privado
│   └─ use-{name}.ts       # hooks TanStack Query
├─ schemas/{name}.schemas.ts
├─ types/{name}.types.ts
├─ i18n/en.json, es.json  # obligatorio
├─ components/
└─ index.ts               # barrel público
```

- **Regla de dependencia:** un feature nunca importa de otro feature
- **components/ui/ nunca importa** de @features/* ni de @store/*
- **El barrel es la única puerta pública:** handler, keys y mock data nunca se exportan fuera del feature
- **Reglas cargadas por path** en .claude/rules/, no solo prosa

Del mock a la API real: una variable de entorno

Ningún componente ni hook sabe de dónde vienen los datos. El único punto de intercambio es la capa de handlers.

```
# .env.local
NEXT_PUBLIC_API_URL=https://api.midominio.com
NEXT_PUBLIC_USE MOCKS=false # era true
```

```
// {name}.handler.ts: el único punto que cambia
if (USE MOCKS) {
  // lee de _mock-data.ts
} else {
  // llama a NEXT_PUBLIC_API_URL
}
// misma firma pública en ambos casos
```

- **Cero cambios** en componentes, hooks o páginas
- **Firma pública idéntica:** el hook llama al handler igual en ambas ramas
- **Validación en frontera** con Zod, en ambas ramas por igual
- **Decisión deliberada:** datos mock por defecto. El template debe funcionar sin infraestructura, y montar un backend solo para evaluarlo sería una barrera desproporcionada

La tesis del método

Todo lo que sigue en este bloque es consecuencia directa de esta idea, no una lista de buenas prácticas sueltas.

Con IA, el cuello de botella deja de ser escribir código y pasa a ser **verificarlo.**

1. Separar decisión de ejecución

Dos conversaciones distintas, roles asimétricos.

2. Acotar en unidades verificables

Bloques con condiciones de cierre no negociables.

3. No aceptar sin comprobar

Ninguna afirmación entra sin poder verificarse.

Dos roles, dos conversaciones distintas

El arquitecto redacta un prompt con estructura fija; el ejecutor lo aplica y devuelve un resumen que el arquitecto verifica de forma independiente.

	ARQUITECTO	EJECUTOR
Decide	qué hay que conseguir y por qué	cómo implementarlo
Toca el repo	no	sí
Contexto	el proyecto entero, el roadmap	un sub-bloque, arranca limpio cada vez
Entrega	un prompt con contexto, objetivos, verificación y condiciones de cierre	código + resumen de cierre
Modelo	Claude Opus	Claude Sonnet

Las dos conversaciones ocurren dentro de la misma sesión de trabajo, pero por separado: el contexto no se satura porque el ejecutor no arrastra las decisiones de hace veinte bloques, y cada decisión queda trazada explícitamente en su prompt. Por eso el reparto de modelo tiene sentido: el rol que decide y verifica necesita el modelo más capaz y el contexto completo; el que ejecuta una tarea acotada no lo necesita, y el coste se concentra donde aporta.

Decisiones fijadas vs. decisiones delegadas

Cada prompt distingue explícitamente entre lo que ya está decidido y lo que el ejecutor debe resolver y justificar. Evita las dos patologías opuestas: microgestionar cada línea, o dejar que el ejecutor improvise arquitectura.

Decisión fijada: ejemplo real

node:22.16-slim, no alpine

next/image depende de sharp, que usa binarios nativos precompilados contra **glibc**. Alpine usa musl, así que sharp falla al instalar o se rompe la primera vez que se pide una imagen. Es un fallo en **runtime, en el VPS**, no en build. `slim` cuesta ~80 MB más y elimina ese modo de fallo por completo.

Decisión delegada: formato

OPCIÓN A: descripción de la alternativa 1

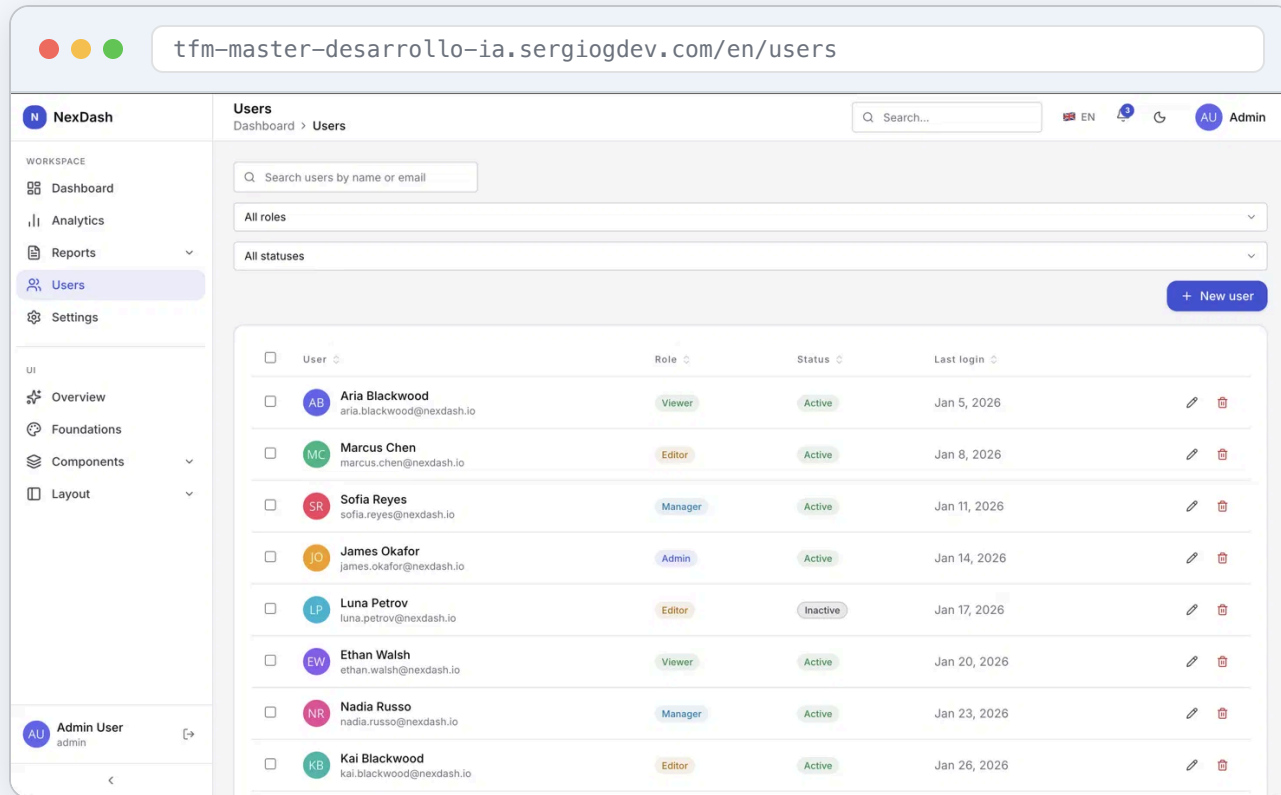
OPCIÓN B: descripción de la alternativa 2

RECOMENDACIÓN: cuál y por qué, salvo que el ejecutor encuentre algo en el código que cambie el cálculo

El ejecutor elige y **justifica su elección en el resumen de cierre**. Sin justificación, la decisión no se acepta.

64 bloques encadenados, cada uno deja el repo desplegable

El trabajo se divide en bloques numerados (B1, B2...), subdivididos cuando mezclan capas (B12.1, B12.2, B12.3). Ninguno se cierra sin pasar sus condiciones.



```
[ ] npm test      → verde
[ ] npm run typecheck → 0 errores
[ ] npm run lint   → 0 errores
[ ] npm run check:docs → sin rutas rotas
[ ] npm run build  → pasa
[ ] documentación actualizada
```

- **Sin ramas largas:** los 49 commits son lineales sobre main, sin estados intermedios rotos
- **Dividir cuando el bloque mezcla capas:** B13 juntaba deuda de UI y hardening de despliegue, verificables de formas distintas
- **La documentación es parte del bloque,** no un extra: cada uno actualiza el CHANGELOG con qué se hizo, por qué, y qué se descartó

Que los tests pasen no prueba nada

Un suite verde puede estar lleno de tests tautológicos. La única comprobación real: romper el código a propósito y confirmar que los tests fallan.

```
# invertir la regla que hace que /reports
# muestre el label del grupo, no el del hijo
sed -i '' 's/=== ancestors.groupHref/!== ancestors.groupHref/' \
  src/lib/route-info.ts

npm test
→ Test Files  1 failed | 16 passed (17)
→ Tests       1 failed | 55 passed (56)

git checkout -- src/lib/route-info.ts
```

- **Un test que no puede fallar** no es cobertura, es decoración
- Aplicado a **cuatro capas distintas**: lógica pura, mapeo de errores del handler, interacción de componente, config de test
- Cada mutación produjo **exactamente un fallo**: cobertura precisa, sin solapamiento
- Comando y salida reproducidos y verificados en este mismo repositorio al generar este documento

Quando la documentación miente, ninguna herramienta se entera

Seis desfases aparecieron en cinco bloques distintos. Cero detectados por lint, build o los 56 tests: ninguna de esas herramientas lee prosa.

El problema

La documentación afirma cosas sobre el código. Nada verifica que sigan siendo ciertas cuando el código cambia debajo.

La solución

`scripts/check-doc-paths.mjs` extrae cada ruta citada en la documentación viva y falla en CI si no existe en disco. Convierte "¿está la doc al día?" en un check binario.

6 / 5

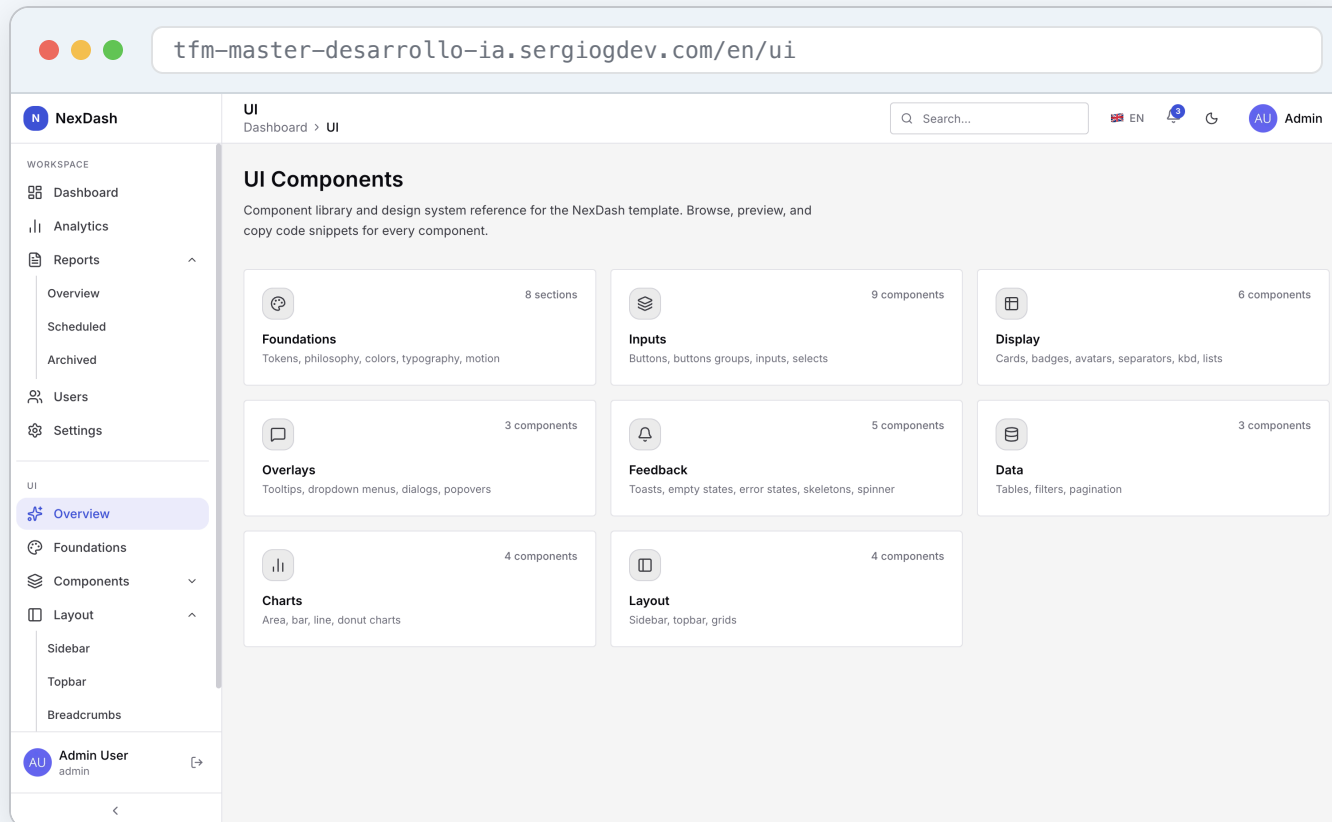
desfases detectados en cinco bloques distintos, cero por herramienta automática previa

8.581 / 19.956

líneas de documentación frente a líneas de código en `src/` (un 43%)

El sistema de diseño se documenta a sí mismo

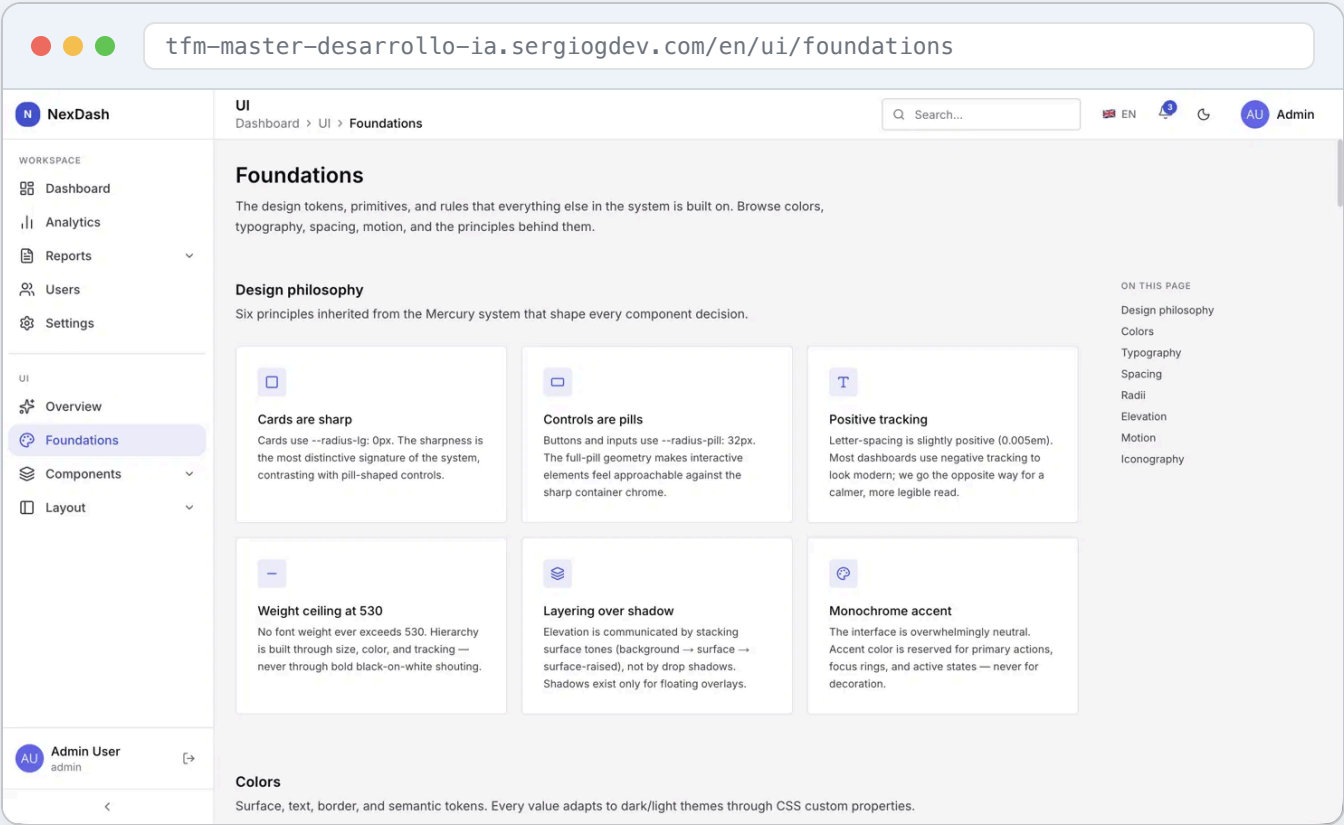
Cada primitiva tiene su página con anatomía, variantes, props y notas de accesibilidad, renderizando el componente real, no una captura estática. Estructura fija en las ~40 páginas de /ui.



- Estructura fija por página: **anatomía → variantes → estados → props → localización**
- El componente que se muestra **es el componente real** importado, no una imagen
- `PropsTable` documenta prop, tipo, default y descripción de cada primitiva
- Cubre más de 30 componentes: botones, formularios, tablas, gráficas, feedback, layout
- `ShowcaseDemo` muestra el snippet copiable junto a la demo funcional traducida al locale activo

Foundations: los tokens, vivos y documentados

Color, tipografía, espaciado, elevación y motion, no en un PDF de marca aparte, sino renderizados desde las mismas variables CSS que usa la aplicación.



- Tailwind v4 sin fichero `tailwind.config`: tokens en `src/styles/tokens.css`
- Dos temas (`.theme-dark` / `.theme-light`) y seis presets de acento vía `[data-accent]`
- Cada valor mostrado en la página es el valor real que consume la app, no una copia
- Arctic Light es el tema claro por defecto de la aplicación

Cuatro fallos reales, trazados en el CHANGELOG

Nada de esto se maquilla. Documentarlo es parte del planteamiento.

QUÉ PASÓ	QUÉ HABRÍA PROVOCADO	QUÉ LO DETECTÓ
Diagnóstico equivocado del ejecutor	El aviso de lockfile se dio por falso positivo; faltaban binarios @next/swc-* de Linux: npm ci habría fallado en el VPS, no en local	El arquitecto reprodujo el build en vez de aceptar el resumen
Checks que no podían fallar	Un assert de CI sobre un campo que nunca está vacío; un test de paginación probado con 5 páginas cuando decía cubrir hasta 7	Verificación por mutación
Dos errores de acotación del arquitecto	Dos bloques de realineamiento documental dejaron fuera justo la parte que fallaba (rutas sin segmento de locale en docs/)	El check automático, en su primera ejecución
Generador de datos correcto, salida absurda	La primera página de usuarios mostraba diez personas con el mismo apellido: el apellido avanzaba cada 10 entradas, justo el tamaño de página	Mirar la salida; ningún test lo habría visto, los datos eran válidos

Lección: los tres fallos que llegaron más lejos eran afirmaciones plausibles que nadie había reproducido.

Qué queda

URL en vivo

tfm-master-desarrollo-
ia.sergiogdev.com

16

documentos en docs/, 8.581 líneas

5

puertas de CI: lint, typecheck, test,
check:docs, build

64

bloques, historial lineal, cero deuda
oculta

DEMO EN VIVO

tfm-master-desarrollo-ia.sergiogdev.com

REPOSITORIO

github.com/SergioGDev/nextjs-dashboard-template

No es que la aplicación funcione, eso se ve en la demo. Lo que importa es que el siguiente proyecto pueda arrancar desde aquí sin arrepentirse.